LidarTreeSegmenter

How it works: from a point cloud to stand volume

A description of all stages, algorithms, parameters and accuracy

This document describes the program as it is in the code. Where something is a simplification or has not been verified yet, it says so plainly. The figures were made on the synthetic test plot that ships with the program (`tree_segmenter_tool/90_make_test_forest.py`, 28 trees); the accuracy figures in chapter 11 come from real mobile laser scans of mountain spruce and beech stands.

How to use the program step by step is in the **User manual** (`docs/user_manual.pdf`).

Contents

1. Introduction: what the program does and what it is made of
2. Glossary: point cloud, voxel, slice and other words
3. Stage 1: tree detection and DBH
4. Stage 2: stem, branches, foliage, shrubs and snags
5. Stage 3: report, cylinder check, choice of DBH, volume
6. Matching to a field measurement (GPS)
7. Scanner path, terrain model and walk-through animation
8. List of algorithms
9. Parameters and stand classes
10. Columns and indicators in Excel
11. Accuracy: what we know from tests
12. Limitations
13. References

1. Introduction

LidarTreeSegmenter takes a point cloud from a laser scanner (a `.laz` or `.las` file) and does with it what a forester does with a tape, a caliper and a height meter: it finds the trees, measures their DBH and height, computes basal area and volume, and finally assembles a stand report. In addition it splits every tree into stem, branches and foliage, tells shrubs and snags apart from healthy trees, removes objects that have no real stem, and, when you have a few trees measured with GPS, moves the whole scan into real-world coordinates.

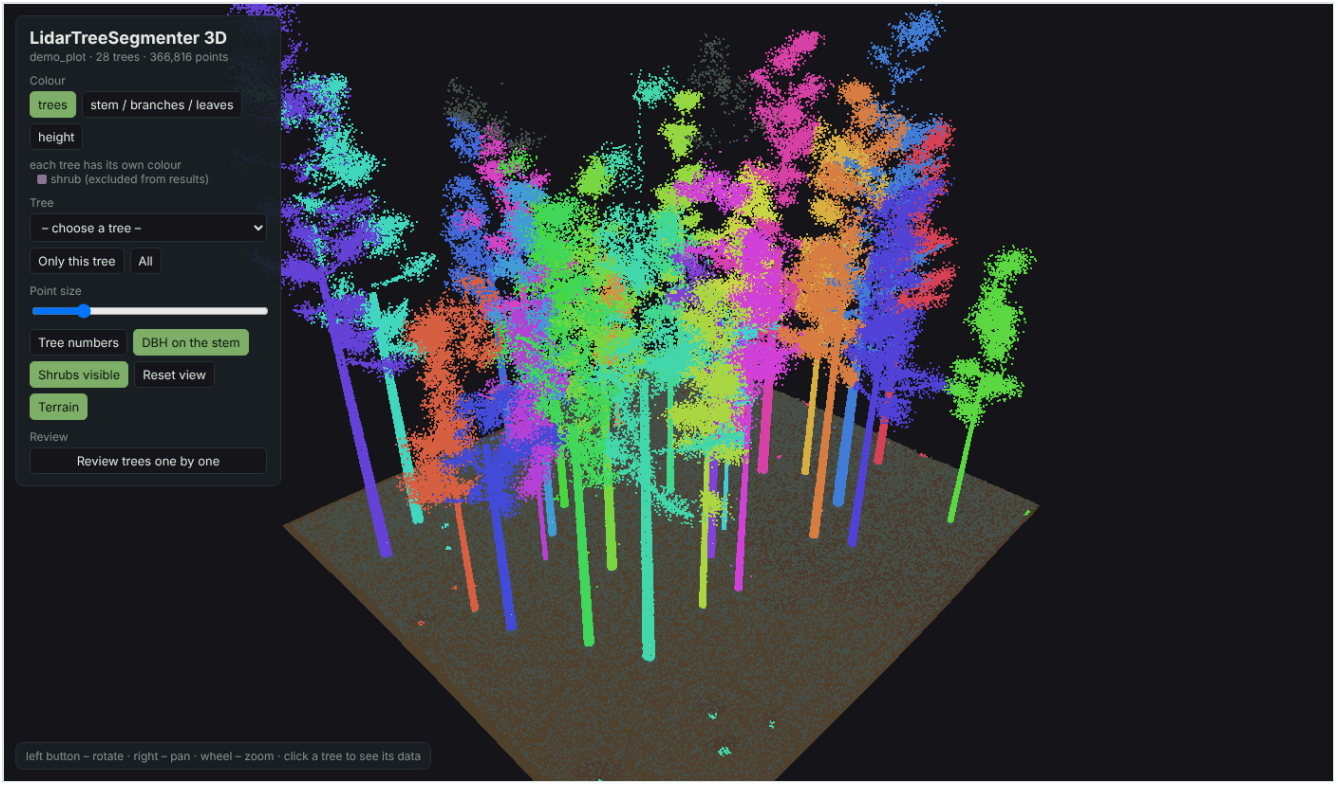


Fig. 1. Point cloud of the test plot (about 570,000 points on 0.09 ha) after segmentation. Each point is one laser reflection from bark, a needle, a branch or the ground; each tree has its own colour.

The program works in three stages. Each stage is a separate file in the `program` folder and each saves its results, so the next one can use them, and the report can be recomputed without processing the cloud again.

Stage 1 – trees

cleaning the cloud, terrain model, stem search in slices, assigning points to trees, DBH and height `05_wykryj_drzewa.py` + `tree_segmenter_tool`

Stage 2 – tree parts

stem tracking up to the crown, shape features, wood or leaf (random forest), shrub / snag, understory, preliminary species `10_pien_galezie_liscie.py`

Stage 3 – report

cylinder check, choice of DBH, height curve, volume, per-hectare indicators, Excel, GeoPackage, field measurement `30_raport.py`

3D view

cloud in the browser, reviewing trees one by one `40_widok_3d.py`

> **The most important thing to remember**
>
> Stage 1 is pure geometry: the program has not “learned” from examples, it looks for shapes that look like a stem (vertical, round, hollow inside). The only machine learning in the whole program is in stage 2, when separating wood from leaves, and it is trained from scratch on every scan without your labels. That is why every result can be explained step by step, which is what this document does.

2. Glossary

Word	Meaning
LiDAR, laser scanner	A device that sends laser pulses and measures how long they take to return. From the time and direction it computes the position of the reflection point (x, y, z).
MLS, TLS	Mobile scanning (you walk with the scanner) and terrestrial scanning from a tripod. With MLS the scanner sees trees from many sides, but not always all around.
Point cloud	A set of millions of points (x, y, z). In the program it is a table with N rows and 3 columns.
.las / .laz	Point cloud file format; .laz is the compressed version. Besides x, y, z the file may have extra fields, e.g. a tree number.
Voxel	A cube in space (a 3D pixel). Dividing the cloud into voxels and keeping one point per cube makes the cloud lighter.
Slice	A thin horizontal layer of the cloud, e.g. 1.25–1.35 m above ground. In a slice a stem looks like a circle or an arc from above.
DTM (terrain model)	The ground surface. The height above ground of every point is measured from it.
Neighbours (k nearest)	The k points closest to a given point. They are found quickly with a k-d tree.
Angular coverage	How much of the stem circumference the scanner saw, in degrees (360° = the whole circumference).
Hollowness	The share of points near the circle that lie away from its centre. A stem is empty inside (the laser reflects from the bark), a shrub is not.
Circle chain	Circles from successive slices stacked above each other, i.e. a piece of stem.
Graph	Points connected by edges to their neighbours, like a road network. It allows computing a “distance along the cloud”.
Random forest	A machine learning method: many simple decision trees vote on whether a point is wood or leaf.
DBH (D1.3)	Stem diameter at 1.3 m.
Slenderness h/d	Height divided by DBH (in the same units). A healthy spruce usually has 50–100.
Snag	A broken tree: thick stem, low height.
Understory	Shrubs and regeneration near the ground that are not part of the measured tree.

3. Stage 1: tree detection and DBH

Stage 1 runs `tree_segmenter_tool/02_detect_trees.py` with settings chosen by LidarTreeSegmenter, and runs the stem search a second time, higher up. The steps appear in the log as `[KROK 1]`, `[KROK 2]` and so on (*krok* = step).

1 Loading and thinning (1 cm voxels)

The program reads the file and turns it into a table of coordinates in metres. A mobile scanner produces very many points in one place (close to its path), so the cloud is thinned first: space is divided into 1 cm cubes and one point is kept from each. The shape of the trees does not change and the computation becomes faster.

> **Algorithm: voxel grid downsampling**
>
> The voxel index is x, y, z divided by 1 cm and rounded down. The program keeps the first point of each voxel (not the mean). Parameter: `VOXEL_WEJSCIOWY = 0.01 m`. Source: Rusu and Cousins 2011 (PCL library).

2 Noise removal (SOR)

Noise means single points “in the air”: reflections from dust, droplets, measurement errors. For each point the program computes the mean distance to its 12 nearest neighbours. A point in a dense crown has neighbours close by, a ghost point far away. Points whose mean distance is larger than the mean of the whole cloud plus 2.5 standard deviations are removed (typically about 3 % of points).

> **Algorithm: Statistical Outlier Removal (SOR) + k-d tree**
>
> Parameters: `SOR_SASIEDZI = 12`, `SOR_PROG_ODCHYLEN = 2.5 σ`. Neighbours are found with a k-d tree (`scipy.spatial.cKDTree`), which divides space into boxes so that not every point has to be compared with every other. The code warns that a too strict threshold (e.g. 1.5) “eats” tree tops and lowers heights. Sources: Rusu et al. 2008 (SOR), Bentley 1975 (k-d tree).

3 Terrain model

The terrain is divided into 0.5 × 0.5 m squares, and in each the lowest point is taken as a ground candidate. Sometimes the scanner did not see the ground in a square and the “lowest point” is on a branch. So the program joins neighbouring squares if their heights differ by at most 40 cm and keeps only the largest connected “island”: that is the ground. It also rejects squares half a metre higher than the median of their 9 neighbours (logs, mounds, shrubs).

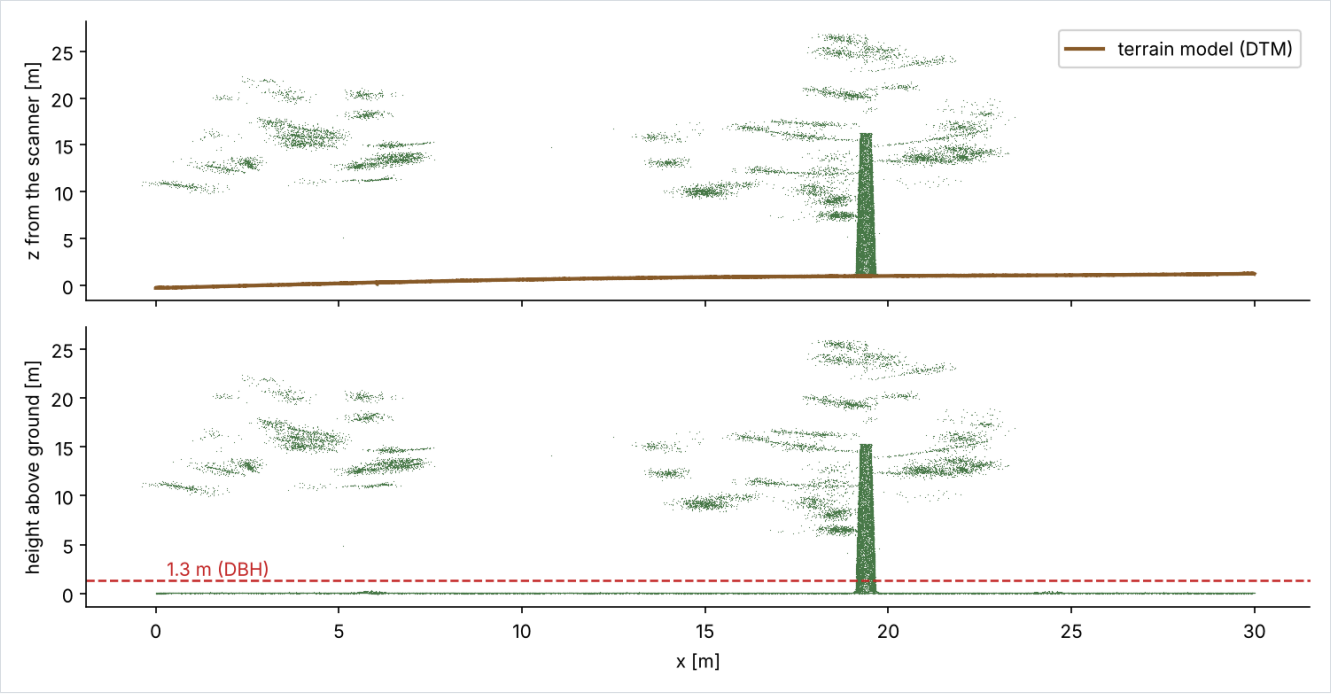


Fig. 2. A strip through the test plot. Top: height z from the scanner; the brown line is the terrain model (the ground rises to the right). Bottom: after subtracting the terrain every point has a height above ground, and the red line at 1.3 m is where DBH is measured.

> **Algorithm: grid minimum + largest connected component + median filter**
>
> A simple method of our own, not the CSF (cloth simulation) filter known from airborne LiDAR: the code says explicitly that CSF is meant for airborne data. Connected components: `scipy.sparse.csgraph.connected_components` (Hopcroft and Tarjan 1973). Parameters: cell 0.5 m, step between cells 0.40 m (with a 0.5 m cell that is a slope of up to 80 %), difference from neighbours 0.5 m.

4 Height above ground

Under every point the program computes the ground height by interpolating between the terrain model points (Delaunay triangles, value weighted by position in the triangle) and subtracts it from z. At the plot edges, where there are no triangles, it takes the height of the nearest ground point. Trees more than 4 m from visible ground get a note that their height is uncertain.

> **Algorithm: linear interpolation on a Delaunay triangulation**
>
> `scipy.interpolate.griddata(method="linear")`, using the Qhull library underneath (Barber et al. 1996, Delaunay 1934).

5 Stem search: tracking slice by slice

This is the most important step of stage 1. The program cuts a band from 0.5 to 6 m above ground and looks for stems in it in four sub-steps.

a) Only vertical surfaces remain. For each point the program takes 20 neighbours and checks how the surface they lie on is oriented (principal component analysis, PCA). Stem bark

is a vertical wall, leaves and branches lie any way. Points whose surface is nearly vertical (vertical component of the normal below 0.25) are kept.

b) Circles in each slice. The band is divided into 25 cm slices. In each slice the vertical-surface points are grouped with DBSCAN: points lying densely next to each other form a group, isolated points are rejected. The program fits a circle to every group and checks three things: whether the circle fits the points well (error up to 4 cm), whether at least 90° of the circumference is visible, and whether the centre is empty (at least 80 % of points lie farther than 0.7 radius from the centre).

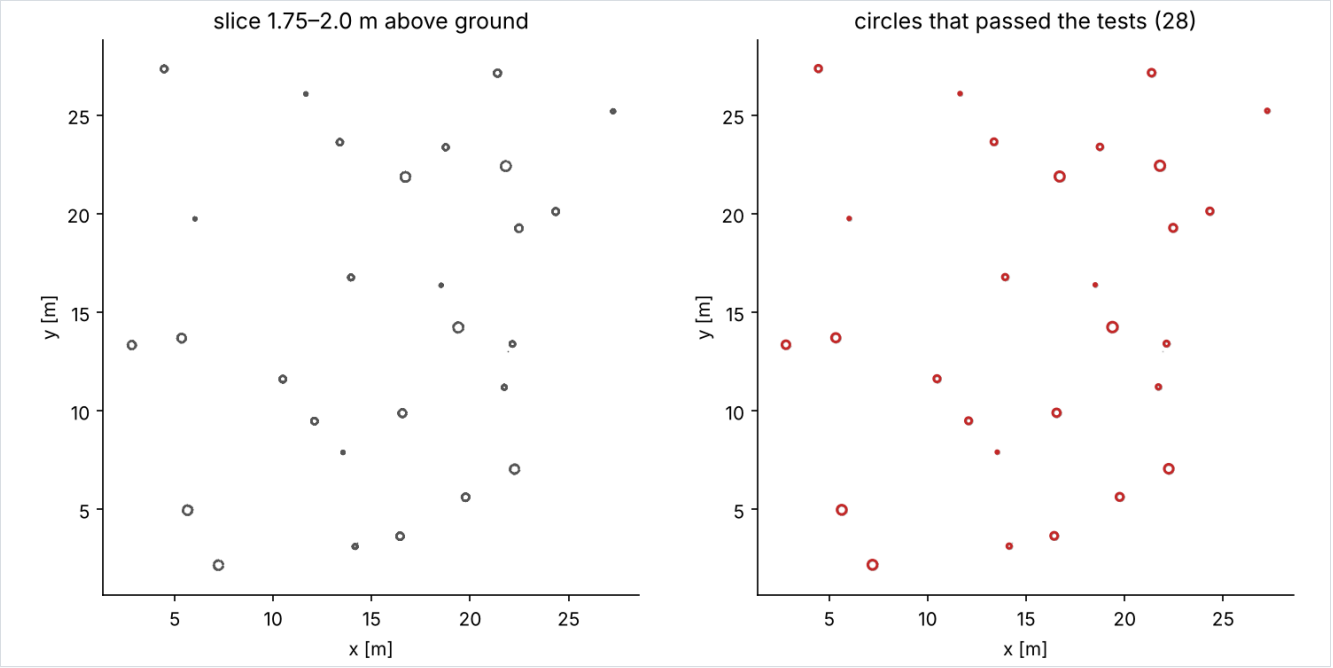


Fig. 3. A slice 1.75–2.0 m above ground of the test plot. Left: the points of the slice. Right: the circles that passed all tests, one per stem.

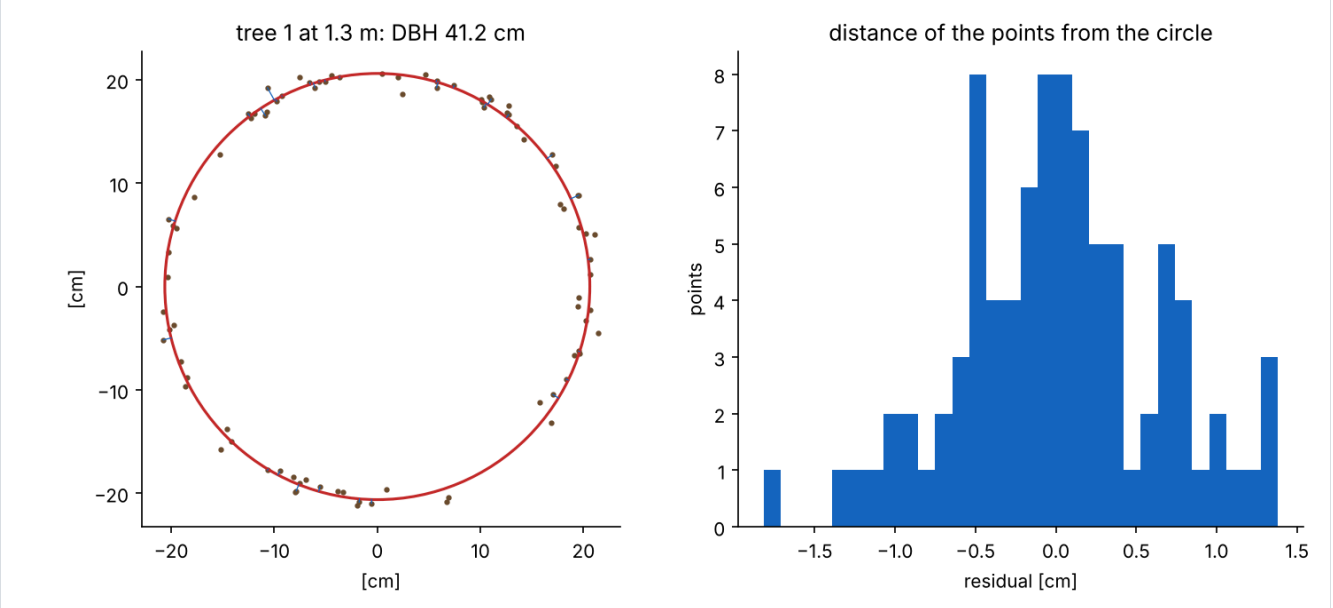


Fig. 4. Circle fitted to the bark points of tree 1 at 1.3 m. Blue lines are the deviations of the points from the circle; the program looks for the circle for which they are smallest. Right: their distribution, within ±1.5 cm.

c) Joining circles into chains. Going from the bottom up, the program attaches a circle to a chain if it lies above it (shift at most 25 cm per slice, allowed gap up to 3 slices) and has a similar radius (change up to 50 %). A chain is a stem. A chain shorter than 6 links (1.5 m) is not accepted as a tree.

d) Re-measurement along the stem axis and DBH. Through the circle centres of the chain the program fits a straight line, the stem axis. Every slice is measured again, but only from points closer than 60 cm to the axis, with RANSAC: 500 times it draws three points, puts a circle through them and counts how many points lie on it (±2 cm), minus a penalty for points inside. The circle centre must lie at most 8 cm from the axis. According to tests recorded in the code this reduced DBH overestimation from +12 cm to +2.6 cm. Chains ending above 1.6 m are extended down to 1.0 m by prolonging the axis and radius.

The DBH is the radius read at 1.3 m from a line fitted to the circles between 0.8 and 3.5 m, times two. A leaning stem (more than 5°) gets a small correction, because the horizontal section of a leaning cylinder is an ellipse. When the chain does not cover 1.3 m (±0.3 m), the DBH stays empty: extending downwards is deliberately switched off, because on a test plot it gave an error of 5.5–11.5 cm instead of 2.8 cm with a normal measurement.

LidarTreeSegmenter runs this step a second time, higher, from 2.5 to 8.5 m. A stem found only higher up that does not overlap any earlier one is added as a new tree (“detected from above”). This helps in dense understory.

> **Algorithms of step 5**
>
> PCA (Hoppe et al. 1992): surface normal from 20 neighbours. DBSCAN (Ester et al. 1996): density clustering, radius 15 cm, min. 10 points. Kåsa circle fit (Kåsa 1976): algebraic least squares with three rounds of outlier rejection (mean + 2σ). RANSAC (Fischler and Bolles 1981): random point triples, with an added penalty for a filled centre and an “anchor” on the stem axis (our own modifications). Chain joining: greedy attachment of the nearest circle (our own). Parameters: slices every 0.25 m, range 0.5–6.0 m, stem diameter 6–120 cm (in a young stand 4.8–40 cm), min. 6 links.

6 Assigning points to trees (graph segmentation)

Once the stems are found, every crown point has to be told which tree it belongs to. The simplest method (nearest stem from above) cuts crowns vertically and overhanging branches go to the neighbour. So the program uses a graph: the cloud is thinned to 30 cm voxels, every voxel is connected to its 20 nearest neighbours (edges up to 1 m), and then for every voxel the shortest path “along the cloud” to some stem is computed. A step sideways costs twice as much as a vertical step, so the path prefers to go down a branch to its own stem rather than jump to a neighbour.

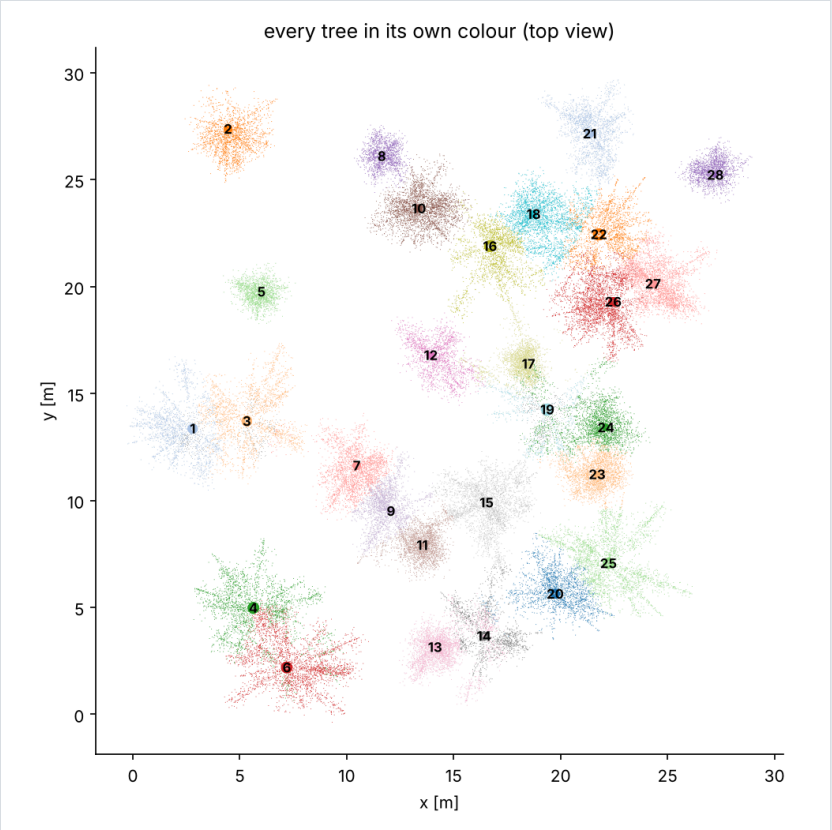


Fig. 5. Result on the test plot, top view: every tree has its own colour and number. Grey points are ground and what could not be connected to any stem.

> **Algorithm: multi-source Dijkstra on a k-nearest-neighbour graph**
> Edge weight = $\sqrt{(4 \cdot (dx^2 + dy^2) + dz^2)}$. Sources are voxels at 1–4 m within 0.8 m of a stem. Unreachable voxels get the number of the nearest reached one (up to 3 m); points lower than 0.3 m above ground get zero. Parameters: `GRAF_ROZMIAR_VOXELA = 0.30 m`, `GRAF_SASIEDZI = 20`, `GRAF_MAX_DLUGOSC_KRAWEDZI = 1.0 m`, `GRAF_KARA_POZIOMA = 4`. In tests recorded in the code this reduced the height error (RMSE) from 3.32 m to 1.06 m compared with the nearest-stem method. Sources: Dijkstra 1959; similar approaches: Tao et al. 2015, Wang 2020.

7 Tree height, dead trees, forks

Tree height is the highest point assigned to the tree, above ground. The program also flags probably dead trees (almost no points in the upper 40 % of the height) and stems that join below 0.5 m (forks). These are hints; they do not remove anything.

8 Sanity filter and saving

Trees with slenderness h/d below 25 (e.g. 5 m height with 40 cm DBH) are put in a separate table `odrzucone_smuklosc.csv`, because on four test plots these were almost always errors. Other filters (upper slenderness, “stem wall”, taper) are off, because they rejected real trees. Finally the cloud with a tree number for every point (`_segmenty.laz`) and the tree table are saved. The plot area is the number of 25 × 25 cm squares containing any point (overestimates by about 2 % at the edge).

4. Stage 2: stem, branches, foliage

Stage 2 takes each tree separately and splits it into parts in more detail. The result is a cloud with the field `czesc` (1 stem, 2 branches, 3 foliage, 4 understory), a table of stem cross-sections every 20 cm, an image of every tree and a GeoPackage.

1 Tracking the stem up to the crown

Stage 1 only looked at the band 0.5–6 m. Now the program looks for the stem of each tree from several starting heights (1, 2, 3, 4, 5 and 6.5 m) and from each start goes up and down with circles every 20 cm, predicting the position of the next circle from the previous ones. The longest consistent chain wins. If the stem base is more than 35 % thicker than the taper higher up suggests, a shrub has stuck to the stem; such a section is cut out. The missing base is then reconstructed: axis and diameter are extended downwards (the stem gets 0–1.5 cm of radius thicker per metre downwards). Reconstructed circles are blue on the images and have a blue background in Excel.

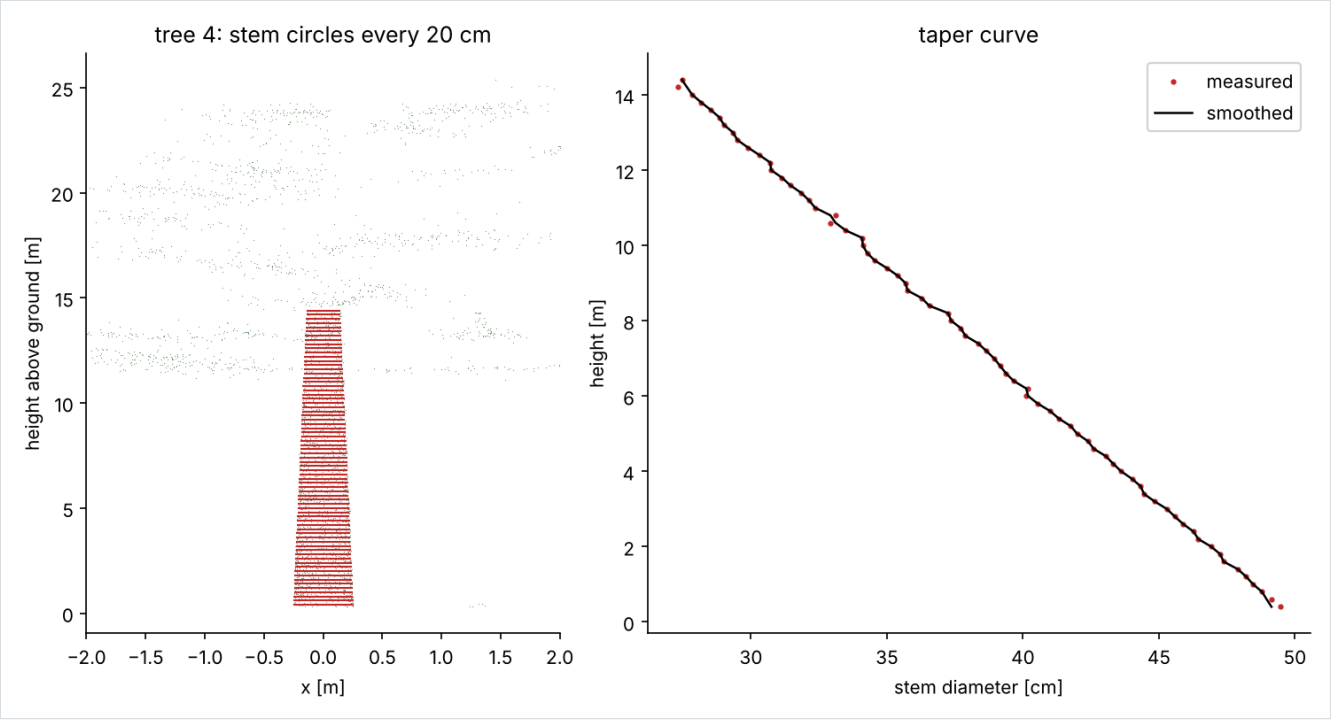


Fig. 6. Tree 4 of the test plot. Left: stem cross-sections every 20 cm (red lines) over the tree's points. Right: diameter as a function of height, the taper curve. The black line is the smoothed values (median of 5 neighbouring sections).

Algorithms

Start: DBSCAN (radius 8 cm) + RANSAC circle (200 draws, band ±2 cm) + Kåsa on the agreeing points. Tracking: position predicted by a line through the last 6 circles, RANSAC “anchored” 12 cm from the prediction; the radius may change by −30 % to +15 % upwards and only by −8 % to +10 % downwards. Smoothing: moving median of 5. The idea of tracking a stem with circles is related to QSM models (Raumonen et al. 2013, Hackenberg et al. 2015), but the program does not implement them.

2 Shape features, and 3 wood or leaf

The tree cloud is thinned to 2.5 cm voxels. For every voxel the program takes its neighbours (12, then 40) and checks what shape they form: a line (branch), a plane (bark seen from close) or a ball (needles, leaves). Numerically this is PCA: the spread of the points gives three numbers $\lambda 1 \geq \lambda 2 \geq \lambda 3$ (like length, width and thickness of the little cloud).

- linearity = $(\lambda 1 - \lambda 2) / \lambda 1$: large for branches,
- planarity = $(\lambda 2 - \lambda 3) / \lambda 1$: large for bark,
- scattering = $\lambda 3 / \lambda 1$: large for needles and leaves,
- verticality: whether the main direction is vertical,
- density: distance to the farthest neighbour.

Then the program creates its own training examples: wood is stem voxels and clear lines (linearity above 0.8 at both scales), leaves are clear balls (scattering above 0.15) farther than 0.5 m from the stem. On these examples it trains a random forest (80 decision trees) and asks it about all the other voxels. The result is a wood probability `p_drewno`: above 0.5 a branch, below foliage. The distance from the stem is not given to the model, so that it does not learn the trivial rule “close to the stem = wood”.

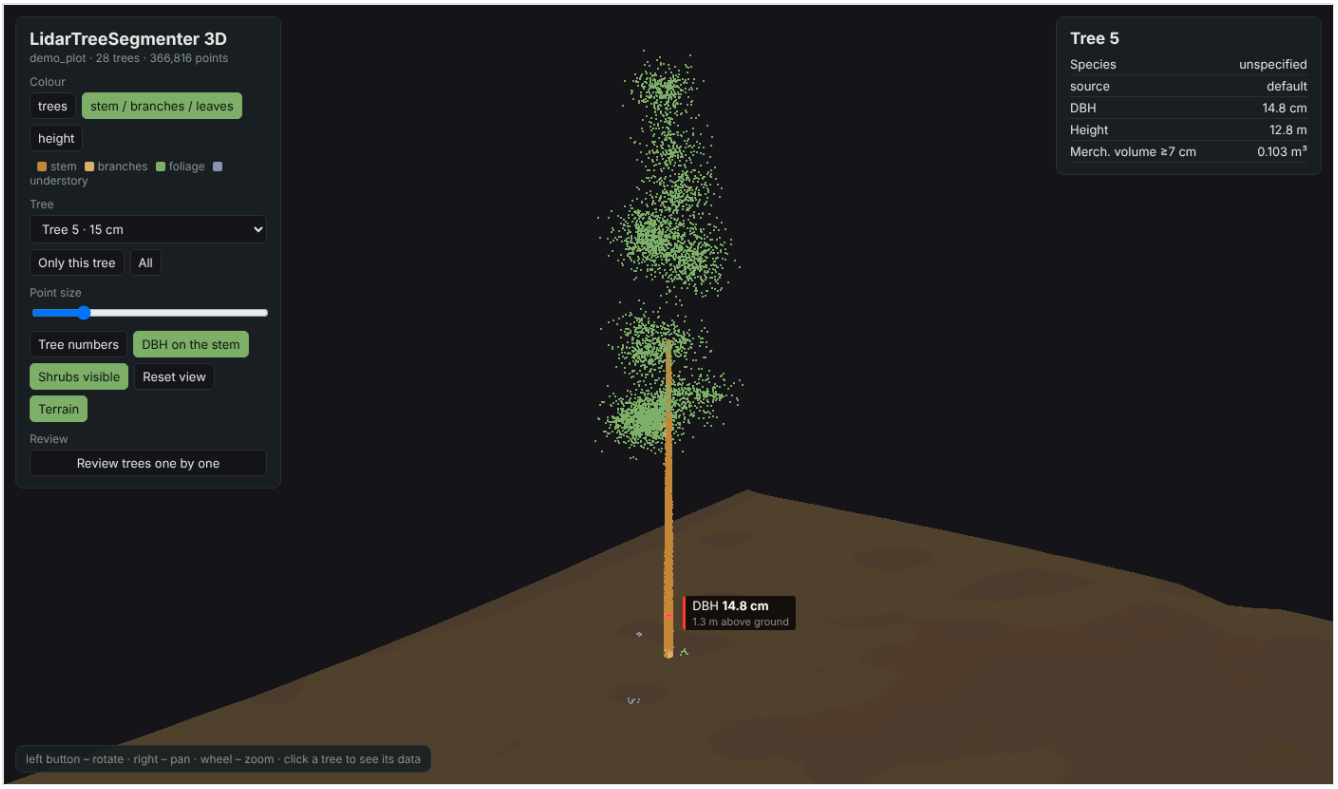


Fig. 7. One tree after splitting into parts, in the 3D view: stem, branches and foliage. The red ring marks the DBH at 1.3 m.

Algorithm: eigenvalue features + random forest trained on rule-based examples

Features: Demantké et al. 2011, Weinmann et al. 2015. Random forest: Breiman 2001 (`RandomForestClassifier`, 80 trees, depth up to 14). Training on automatically chosen examples is self-training (Yarowsky 1995), here in one round. Similar methods: Krishna Moorthy et al. 2020, Wang et al. 2020 (LeWoS), Vicari et al. 2019.

Note

The branches / foliage split has not been verified numerically yet, because there is no cloud with manually labelled wood and leaves. It was only judged visually. In a leaf-off (winter) scan “foliage” in practice means fine twigs.

5 Crown clean-up

Small pieces of “wood” floating in the crown, not connected to the stem (fewer than 25 voxels, about 60 cm of twig), are changed to foliage. The program joins neighbouring wood voxels into groups (connected components of a graph) and checks the size of each group.

Stem check: tree, shrub or snag

A stem in cross-section is empty inside, because the laser reflects from the bark. A shrub or a cluster of branches has points inside too. At 1.3 m and again at 3 m the program computes the share of points near the circle that lie closer to the centre than 0.7 radius. Stems have 0–2 % such points, shrubs 20–50 %.

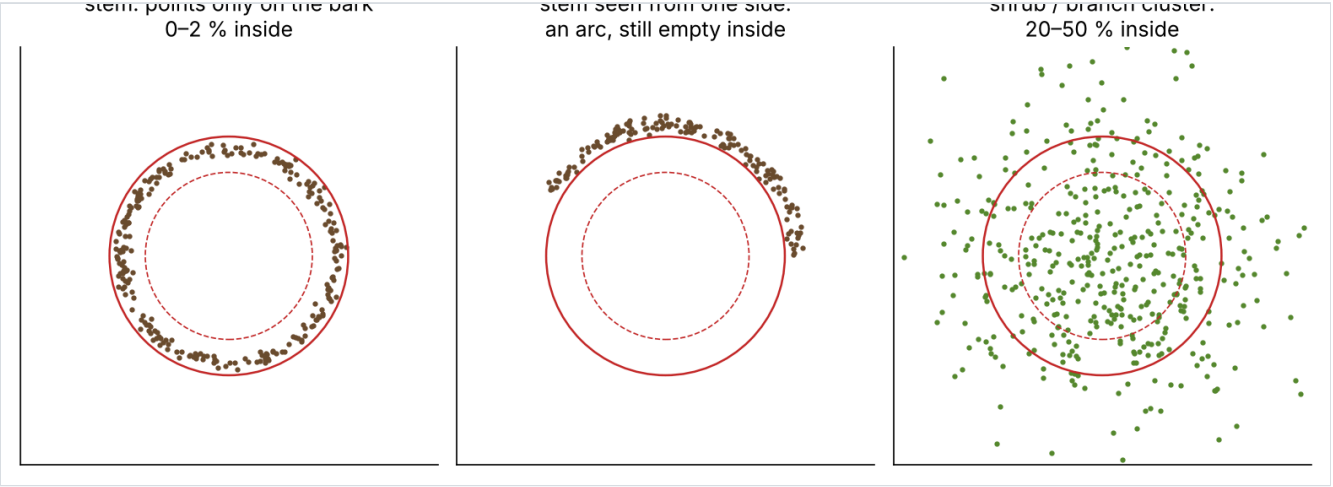


Fig. 8. The hollowness test. The dashed circle is 0.7 × the radius; the share of points inside it decides. A stem seen from one side only is still empty inside, so partial visibility alone does not reject a tree.

Situation	Result	Next
Low (below 15 m) and very thick (h/d below 20), stem clearly hollow at 1.3 and 3 m	snag	stays in the results; volume = volume of the measured stem, not the formula

Low and very thick, stem not hollow	shrub (stump, cluster)	left out of totals
Points inside at both heights and lower than 5 m (young) / 10 m (mature)	shrub	left out of totals
Many points inside, but the tree is tall	uncertain	stays in the results with a note
Stem hollow	ok	–

These are the program's own rules (thresholds: hollow below 15 % of points inside, full from 20 %, snag below 8 %). You have the last word in the tree review mode of the 3D view. In stage 3 a stricter cylinder check follows (chapter 5).

Understory

Branch and leaf points of a tree that lie low (up to 1.5 m in a young stand, up to 3 m in a mature one) and farther from the stem than 0.6 m / 1 m are shrubs near the stem, not branches of the tree. They get a separate class “understory”.

Shrubs as objects

The stage 1 segmentation assigns every vegetation point to some tree, so shrubs were not separate objects, only understory points near trees. The module `lts_krzewy.py` (run in stage 3, so it works after “Recalculate report”) turns them into countable shrubs:

1. **Candidates:** understory points 0.3–4 m above ground, and low branches and leaves of trees that have an empty band of at least 1 m above them before the crown. Stem points and anything closer than 15 cm to the bark are never a shrub, so DBH and stem do not change.
2. **Shrub body:** a 25 cm grid from above; a cell is shrubby when points occupy at least 3 layers of 25 cm between 0.5 and 3.25 m and the highest is at least 0.8 m. Grass and ferns alone (one thin layer) drop out.
3. **Shrubs:** morphological closing and opening, joining neighbouring cells into patches, splitting large patches by their tops (local maxima of the smoothed height at least 1.25 m apart), rejecting patches below 0.5 m².
4. For every shrub: centre, area from above, height (95th percentile), highest point, number of points. Output: `03_TABELE\<scan>_krzewy.csv` and the grid `_krzewy.npz` (used by the 3D view to colour points).

The thresholds were set on one plot and should be checked on others.

Two stems in one tree

When stems stand right next to each other or are fused at the base, the stage 1 segmentation merges them into one tree. Before stem tracking, stage 2 does the following for each tree:

1. Track the main stem (as usual), then remove the points within 3 cm of its bark and track a second time on the rest.

2. The second stem is real when: the measured section is at least 4 m long and at least 40 % of the tree height (dense spruce whorls produce short “stems” made of branches), the diameter is at least 10 cm, lean up to 20° from vertical, median circle arc at least 150°, and at least 80 % of the circles lie beside the main stem (bark does not overlap bark). Branches drop out because they are shorter and more oblique.
3. **The 1.3 m rule:** the tree is split when the second stem was measured from 1.8 m downwards, or the arc of its bark is visible at 1.3 m (at least 60°) next to the main stem, or it stands at least 0.5 m away from the start (base hidden). Otherwise it is a fork above 1.3 m: one tree with a note, as forest mensuration rules require.
4. **Splitting the points:** every point goes to the stem whose bark is closer horizontally; above and below the measured sections the axes are extended by a straight line. The old number stays with the thicker stem (it owns the stage 1 DBH), the second gets a new number and the stage 2 DBH.

Tracking has a random element (RANSAC), so each tree gets up to 3 attempts with a seed derived from the tree number, which makes the result repeatable. Stage 2 takes about 3–4 minutes longer because of this. Limitations: crowns are split only along the extended axes, so the boundary in the crown is approximate; a second stem hidden in dense understory along its whole length will not be found. Output: `03_TABELE\<scan>_rozdzielone.csv`; to switch off: option `--bez-rozdzielania`.

Species (preliminary)

From the shape the program estimates whether a tree is coniferous (spruce) or broadleaved (beech). It weighs four features: how high the straight stem reaches relative to the tree height (50 %), how narrow the top is (20 %), where the crown's centre of mass is (15 %) and the ratio of crown width to length (15 %). Trees lower than 8 m and ambiguous ones stay “unspecified”. These are rules without learning, and in a dense stand crowns overlap, so in the window you can choose “spruce only” or “beech only”.

Scanner tags. A handheld scanner records the time, the chosen code and its own position in the cloud's coordinates each time the marker button is pressed (file `markPoint.csv` in the raw session folder). In tests most tags lay 0.05–0.25 m from a stem found in stage 1, so the tag position is practically the tree position. The module `lts_etykiety.py` assigns each tag to the nearest stem closer than 1 m. When one tree has two tags, the later one wins (treated as a correction). When a plot has several sessions (several folders, each possibly with its own coordinate system), the session whose tags most often hit stems is used, and a session hitting less than half is rejected. Codes: FI spruce, BU beech, TA fir, KI pine, LA larch, EI oak, AH sycamore, ES ash, KR cherry, UL elm, taxus yew.

5. Stage 3: report

Cylinder check: is there really a stem here?

Stage 1 sometimes reports a “tree” where there is none: a cluster of branches, a shrub or a piece of a neighbour's crown that happened to produce a few circle-like arcs. The module `lts_walec.py` (*walec* = cylinder) checks every tree against all scan points, not only those assigned to the tree. A stem is a cylinder: a straight, nearly vertical axis, a constant or gently decreasing radius, points on the bark and nothing inside.

For each tree the program takes all points within 0.7 m of the stem position at 0.6–3.0 m above ground and:

1. in every 25 cm slice looks for a circle with RANSAC; a circle is better the larger the part of the circumference covered by points and the fewer points lie inside it;
2. checks whether the circles of successive slices stack into one cylinder (centre in almost the same place, similar radius);
3. fits a single cylinder to the agreeing slices (the axis may lean slightly, the radius may change linearly with height);
4. measures how well the points fit this cylinder: in how many slices the cylinder has bark points (continuity); what part of the circumference is visible, also compared with the sides the scanner path passed (if the scanner walked around the stem and points are only on one side, it is not a stem); how many points lie inside the cylinder (a shrub has a full centre); and whether the radius is not larger than the spread of the points it rests on (an arc of 100° allows fitting a circle of almost any size).

Result: **stem** (cylinder confirmed), **no stem** (the tree is rejected and not included in totals) or **uncertain** (stays, with a note), plus a DBH from the cylinder at 1.3 m. When two reported trees turn out to stand on the same cylinder, the duplicate is marked. The verdicts are saved in `03_TABELE\<scan>_kontrola_walca.csv` and computed again only when the inputs change. On a real scan of a dense stand this check removed the false trees that the previous version still counted; on the synthetic test plot all 28 trees were confirmed as stems.

Where the DBH in the report comes from

DBH is measured by two independent methods: stage 1 (band 0.5–6 m) and stage 2 (tracking the whole stem). The report chooses in this order:

1. By default the stage 1 DBH.
2. Trees known only to stage 2 are added (so that snags and shrubs do not silently disappear).
3. Your decisions from the tree review override the stem checks.
4. When stage 1 did not measure the DBH (stem hidden at 1.3 m) but stage 2 did, and the tree has at least 8 m and slenderness 40–150, the stage 2 DBH is used. A snag without a stage 1 DBH also gets the stage 2 DBH.
5. When the two methods differ by more than 30 %, the height curve decides.

Height curve

In a stand, height increases with DBH in a predictable way. The program fits the Näslund curve $h = 1.3 + d^2 / (a + b \cdot d)^2$ to the trees for which both methods give the same DBH (±15 %), at least 8 trees. When the methods disagree, the DBH that fits the curve better (by at least 2 m) wins. A tree whose DBH does not fit its height at all (deviation above 6 m or 2.5 times the curve error) gets the note “check the image”.

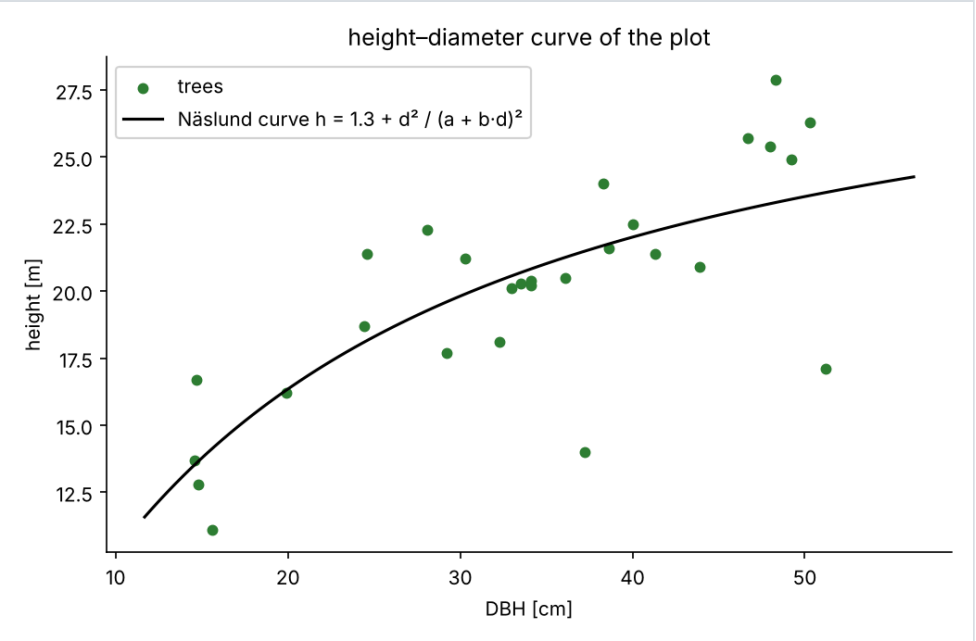


Fig. 9. Height–diameter curve of the test plot (Näslund model fitted to the trees).

> **Algorithm: non-linear least squares (Levenberg–Marquardt)**
>
> `scipy.optimize.curve_fit`. Sources: Näslund 1936; Levenberg 1944; Marquardt 1963.

Volume

The merchantable volume of a tree is computed with the formula $V = g \cdot h \cdot f$, where $g = \pi \cdot d^2/4$ is the cross-section area at 1.3 m, h the height and f the breast-height form factor (what part of a cylinder with cross-section g and height h is merchantable wood). The values in `tablice_miazszosci.csv`: spruce 0.47, beech 0.51, unspecified 0.47, coniferous 0.45, broadleaved 0.50 (the last two for species without their own row, e.g. larch or sycamore). These are approximate values; you can replace the file with regional tables, also with a power formula $V = a \cdot d^b \cdot h^c$.

A second number, “volume of the measured stem section”, is the sum of truncated cones between consecutive stem cross-sections every 20 cm: the volume of the part of the stem the scanner really saw (without the top). It is used as a check, and for snags it replaces the formula, because the formula with height would overestimate. Truncated cone between cross-sections with areas A1 and A2, Δh apart: $V = \Delta h \cdot (A1 + A2 + \sqrt{(A1 \cdot A2)}) / 3$.

> **Why DBH matters so much**
>
> Volume grows with the square of DBH: a tree twice as thick has four times as much wood, so a DBH error of 2 cm at 30 cm gives a volume error of about 13 %.

Stand indicators

Indicator	How it is computed
N/ha	number of trees (without shrubs) / area in ha
Dg, quadratic mean DBH	√(mean of d^2): diameter of the tree with the mean basal area
Lorey's height	Σ(g·h) / Σg: mean height weighted by basal area (thicker trees weigh more)
G/ha	sum of cross-section areas / area
V/ha	sum of volumes / area
DBH distribution	4 cm diameter classes: number of trees, N/ha, G/ha, V/ha

6. Matching to a field measurement

The scanner does not know where north is or where on the map it stood. If you measure a few trees in the field (position from GPS and DBH with a tape), the program finds them in the scan and rotates and shifts the scan into real coordinates.

1. GPS coordinates (degrees) are converted to metres in UTM or PUWG 1992 (Gauss–Krüger projection, Krüger series; accuracy checked against the pyproj library: below 0.1 mm).
2. For every pair of field trees and every pair of scan trees with similar DBH and similar distance between them, the program creates a hypothesis: a rotation and shift that puts one pair on the other.
3. Every hypothesis is scored: for each field tree the nearest scan tree is found; cost = (distance / 5 m)² + (DBH difference / 4 cm)². A tree without a partner costs 9.
4. The 60 best hypotheses are refined with the Kabsch method (best rotation and shift for pairs of points) and re-pairing, as in the ICP algorithm.
5. If there is a second, differently rotated (more than 25°) hypothesis that is almost as good, the result is rated “uncertain”.

How accurate it is

A test with 5 trees and 40 random trials at different GPS errors: with a 2 m error and trees spread 6–14 m from the centre the rotation was correct (error below 15°) in 39 of 40 trials, median error 3.5°. With trees clustered near the centre: 32 of 40. With a 5 m GPS error: 12 and 6 of 40 respectively. Therefore: average the GPS position and choose trees that are spread out and have different DBH.

> **Algorithms**
> Kabsch 1976 (rotation via SVD), Umeyama 1991, ICP: Besl and McKay 1992, projection: Krüger 1912, Karney 2011.

7. Scanner path, terrain model and walk-through animation

A handheld (SLAM) scanner computes its position at every moment and saves it as a path (trajectory): a series of x, y, z points, often with time. The path is in the same coordinate system as the cloud, because the cloud is built from these positions. The program does not need it, but when it is there, it knows from where every tree was seen.

Finding and reading the file

1. **Search** (module `lts_sciezka.py`): first the file chosen in the window, then `01_RAPORT\<scan>_sciezka.*`, then files with traj, sciezka, path, odom, pose, track in the name and with the scan code (from `_N0h__r25` the code is `n0h`), next to the scan and one folder up.
2. **Reading:** LAS/LAZ through laspy (GPS time, if it changes), PLY text and binary (own header reader), text files: comments skipped, separator (semicolon, tab, comma, spaces) and decimal comma detected, columns by header name (`x`, `px`, `pos_x`, `easting`, `field.pose.pose.position.x` and others). Without a header: 12 columns is the KITTI format (3×4 matrix, position in columns 4, 8 and 12), an increasing first column is time (TUM format), otherwise the first three columns are x y z.
3. **Check:** the path must pass through the scan area (with a 30 m margin). Otherwise it is in another coordinate system and the program skips it.

Visibility of a tree from the path

For every stem the program computes the smallest horizontal distance to the path and the coverage: it divides the circle around the stem into 36 sectors of 10° and marks a sector if any path point lies in that direction closer than 15 m. Coverage is the percentage of marked sectors. A tree passed on one side has about 30–50 %, a tree walked around 100 %.

Why: a DBH from a circle fitted to an arc seen from one side only is less certain (a 90° arc allows many different circles), and points far from the scanner are noisier. Trees farther than 12 m from the path are counted separately in the summary. The program does not remove them automatically, because with good visibility they may be measured well. It is information for the tree review.

Terrain model

1. The ground height under every cloud point is z − (height above ground from stage 1). The mean of these values in 0.25 m cells gives a grid (module `lts_dtm.py`). Holes up to 1 m are filled from the nearest cell (distance transform).
2. Slope and aspect are computed from derivatives of the grid smoothed with a Gaussian filter (σ = 0.75 m, ignoring empty cells). Without smoothing small steps gave slopes above 45° on flat ground. Slope = $\arctan(\sqrt{((\partial z/\partial x)^2 + (\partial z/\partial y)^2})$, aspect = direction the slope faces, from north.

3. The dominant aspect in the summary is a circular mean weighted by slope (flat places have almost no influence), rounded to 8 directions.
4. Hillshade (light from the north-west at 45°) and contours (marching squares from matplotlib) come from the same grid. Contour interval: 0.25 m with relief below 3 m, 0.5 m below 10 m, otherwise 1 m.
5. Saving to GeoPackage without extra libraries: rasters as 256 × 256 PNG tiles (GeoPackage tiles standard), elevation and slope with the 2D gridded coverage extension (16-bit PNG with scale and offset), so QGIS shows real values in every pixel.

GeoPackage and QGIS

1. **Thinning slice points.** Every point goes into a 1 cm cell (part “stem”) or a 4 cm cell (branches, leaves, understory) and one point is kept per cell. This is an ordinary 2D voxel grid, computed separately for each part, so bark does not mix with branches. 1 cm is smaller than the noise of a handheld scanner (1–2 cm), so the fitted circle does not change; the bark arc stays continuous and you can see what part of the circumference the scanner covered.
2. **Distance from the circle** (`odl_od_okregu_cm`) is |distance of the point from the centre − radius|. Bark points usually have 0–2 cm; many points a few centimetres from the circle mean a branch, a shrub at the stem or an irregular stem.
3. **Coordinate system.** GeoPackage allows an “undefined Cartesian” system (srs −1), but QGIS treats it as degrees in a project: the scale came out around 1:30,000,000 and layers hidden by scale disappeared. So a scan without a coordinate system gets its own engineering system LOCAL_CS in metres (identifier 990001). A scan with an EPSG system keeps it.
4. **Styles** are saved in the `layer_styles` table (the standard QGIS way of a default style in a GeoPackage) as QML text. Trees: categories by species, size max(1.4; DBH/18) mm. Contours: rules by the field `glowna`. Slice points: visible only from scale 1:400. Rasters: elevation and slope with a colour ramp.
5. The **.qgs project** is an XML file with paths relative to the `.gpkg`, so it works after moving the whole folder. The map extent is the extent of trees and crowns plus 3 m.

Walk-through animation

The 3D view gets the path simplified to a point every 0.25 m. For every 1 m cell of the cloud the program finds the first path point closer than 10 m (k-d tree) and orders the cloud points in that sequence. The browser then shows only the beginning of the point array, longer and longer as the scanner moves, so the animation is smooth even with 600,000 points. This is a simplification: a real scanner sees farther than 10 m, but sees hidden places later, so 10 m reflects the scanning order well.

> **Algorithms**
> Visibility sectors: our own. Distance transform: Danielsson 1980. Weighted Gaussian filter (normalized convolution): Knutsson and Westin 1993. Slope and aspect: Horn 1981. Contours: marching squares, Lorensen and Cline 1987. k-d tree: Bentley 1975.

8. List of algorithms

Algorithm	Where	What for	Source
Voxel grid	stage 1 step 1, stage 2, graph, slice points in GeoPackage	thinning the cloud	Rusu and Cousins 2011
Statistical Outlier Removal (SOR)	stage 1 step 2	noise removal	Rusu et al. 2008
k-d tree	everywhere	fast neighbour search	Bentley 1975
Grid minimum + connected components	stage 1 step 3	terrain model	own; Hopcroft and Tarjan 1973
Interpolation on a Delaunay triangulation	stage 1 step 4	height above ground	Delaunay 1934; Barber et al. 1996
PCA (normals, eigenvalue features)	stage 1 step 5, stage 2 step 3	vertical stem walls; neighbourhood shape	Hoppe et al. 1992; Demantké et al. 2011; Weinmann et al. 2015
DBSCAN	stage 1 step 5, stage 2 step 2	grouping points in a slice	Ester et al. 1996
Kåsa circle fit	stages 1 and 2	stem diameter	Kåsa 1976
RANSAC (with centre penalty and axis anchor)	stages 1, 2 and 3	circles robust to branches and shrubs	Fischler and Bolles 1981
Joining circles into chains	stage 1 step 5	stem from circles	own
Cylinder fit to stacked slice circles	stage 3, cylinder check	removing false trees, DBH from the cylinder	own; RANSAC: Fischler and Bolles 1981
Jaccard similarity	stage 1 (“stem wall”)	diagnostics	Jaccard 1912
Multi-source Dijkstra on a k-NN graph	stage 1 step 6	assigning points to trees	Dijkstra 1959; Tao et al. 2015
Random forest, self-training	stage 2 step 4	wood or leaf	Breiman 2001; Yarowsky 1995
Connected components	stage 2 step 5	crown clean-up	Hopcroft and Tarjan 1973
Rules with voting	stage 2	species group, stem check, understory	own

2D occupancy grid + height layers, morphology, component labelling, split by local maxima	stage 3, shrubs	shrubs as objects	own; morphology: Serra 1982
Second stem tracking after removing the main stem	stage 2, before stem tracking	splitting fused trees	own; RANSAC: Fischler and Bolles 1981
Näslund curve, Levenberg–Marquardt	report	choice of DBH, check	Näslund 1936; Marquardt 1963
V = g·h·f, truncated cones	report	volume	Husch et al. 2003; West 2009
Kabsch, ICP, Krüger series	field measurement	matching and coordinates	Kabsch 1976; Besl and McKay 1992; Krüger 1912
Visibility sectors from the path	scanner path	which side of the stem was seen	own
Weighted Gaussian, grid derivatives, marching squares	terrain model	slope, aspect, hillshade, contours	Knutsson and Westin 1993; Horn 1981; Lorensen and Cline 1987
IoU + Hungarian algorithm	method comparison (program 20)	segmentation evaluation	Kuhn 1955

9. Parameters and stand classes

The stand class changes several parameters at once. The numbers are in `klasy_drzewostanu.csv` (open in Excel or Notepad). The “automatic” class changes nothing in stage 1, and in stage 2 chooses the class from the median DBH (below 20 cm: young).

Parameter	Young	Mature	What it does
Smallest stem diameter	4.8 cm	6 cm	thinner circles are not a stem (stage 1)
Largest stem diameter	40 cm	120 cm	thicker point groups are not a stem (stage 1)
DBH threshold in the report	7 cm	7 cm	thinner trees are left out of the results
Shrub height	5 m	10 m	full inside and lower = shrub
Understory up to height	1.5 m	3.0 m	lower branches away from the stem = understory
Understory from distance	0.6 m	1.0 m	as above
Farthest from the stem (simple method)	5 m	8 m	only a fallback when the graph does not work

Main fixed parameters

Parameter	Value	Where
Input voxel	1 cm	stage 1 step 1
SOR: neighbours, threshold	12, 2.5 σ	stage 1 step 2
Terrain model cell	0.5 m	stage 1 step 3
Stem search	0.5–6 m every 0.25 m, second pass 2.5–8.5 m	stage 1 step 5
Circle: error, coverage, hollowness	≤ 4 cm, ≥ 90°, ≥ 0.80	stage 1 step 5
Min. stem length	6 slices = 1.5 m	stage 1 step 5
Graph: voxel, neighbours, edge, horizontal penalty	0.3 m, 20, 1 m, ×4 (i.e. ×2 distance)	stage 1 step 6
Min. slenderness	25	stage 1 filter
Stem cross-sections	every 20 cm	stage 2
Feature voxel, neighbours	2.5 cm; 12 and 40	stage 2
Wood threshold	0.5	stage 2

Cylinder check: band, slice, search radius	0.6–3.0 m, 0.25 m, 0.8 m	stage 3
Form factors	spruce 0.47, beech 0.51	report
GPS: position σ, DBH σ	5 m, 4 cm	field measurement

10. Columns and indicators in Excel

The report `<scan>_REPORT.xlsx` has the sheets Summary, Trees, (Shrubs), (Field measurement), Settings and Explanations. Columns of the Trees sheet:

Column	Meaning
Tree no.	the same number as in the clouds, on the images `drzewo_NNN.png` and in the 3D view
X, Y [m]	stem position in the scan coordinate system
DBH D1.3 [cm]	DBH after the choice (chapter 5); g and V are computed from it
Tree height H [m]	highest point of the tree above ground
Basal area g [m²]	$\pi/4 \cdot D^2$
Merchantable volume V [m³]	formula of the species; for a snag the measured stem
Slenderness h/d	height / DBH
Distance from scanner path [m]	smallest horizontal distance of the stem from the path (when there is a path file)
Path coverage [%]	share of the 36 sectors around the stem in which the path was closer than 15 m
Detected from above	“yes”: stem found only in the search at 2.5–8.5 m
Reviewed manually	“yes”: your decision from the 3D view
Species, Species – source	species for the volume formula and its source: default / detected (preliminary) / whole plot / scanner tag (code) / entered manually / reviewed manually
Species group confidence [%]	how clearly the rules pointed to the group
Crown base height [m]	height at which the crown begins
DBH notes	explanations: uncertain (part of the circumference visible), shrub, snag, no stem, DBH from stage 2, check the image…
Stage 2 DBH and its source	the second, independent DBH (measured or reconstructed), for comparison
Stem visible from/to, base reconstructed from/to	extent of the measured and reconstructed stem
Volume of measured stem section	sum of truncated cones, without the top

Stem / Branches / Foliage [% pts]	share of the tree's points in each class
Number of points	how many cloud points belong to the tree

Colours: blue background = value from a reconstructed stem base; yellow = there is a DBH note. Shrubs and objects without a stem are grey below the table and are not included in totals.

11. Accuracy: what we know from tests

Tree detection (segmentation) against a manual reference

A tree counts as found when its points overlap a reference tree by at least half (IoU ≥ 0.5). Recall says how many real trees were found, precision how many of the found ones are real trees. Two manually segmented mobile laser scans of mountain forest were used.

Plot	Method	F1	Recall	Precision	Mean IoU
A (36 trees)	stage 1 (tree_segmenter_tool)	76.7	63.9 %	95.8 %	0.81
A	3DFin (for comparison)	63.0	47.2 %	94.4 %	0.79
B (113 trees incl. regeneration)	stage 1 (tree_segmenter_tool)	61.5	53.1 %	73.2 %	0.69

The program rarely invents trees but misses some real ones; most of the “missed” ones are found, only the crown is mixed with a neighbour (IoU 0.3–0.45). Of the 84 trees taller than 15 m on plot B, 83 % have IoU ≥ 0.3. Regeneration (low trees) is almost not detected (10 %).

DBH and height

- DBH measured on plot B: RMSE 2.8 cm (recorded in the code).
- Stage 2 found the stem in 35 of 36 trees (A) and 105 of 113 (B). Stage 2 DBH agrees with stage 1: median difference 1.9 cm (52 trees on B); about 6 trees differ by more than 15 cm, and these are now decided by the height curve.
- RANSAC anchor on the stem axis: DBH overestimation +12 cm → +2.6 cm.
- Graph segmentation: height RMSE 3.32 m → 1.06 m, mean overestimation +0.94 m (overhanging branches of neighbours).
- Area from the 25 cm grid: overestimation about 2 %.
- Synthetic test plot (sanity check only, it is much easier than a real forest): all 28 trees found, DBH within ±0.2 cm of the true value.

> **Not verified yet**
>
> The branches / foliage split, species recognition and form factors on trees measured in the field. When you have a control measurement (DBH with a tape, height with a height meter, species), the “Field measurement” sheet shows the differences and the parameters can be tuned.

12. Limitations

- **Interlocking crowns:** graph segmentation leaves about +0.9 m height overestimation on average; machine learning methods trained on labelled trees would do better.
- **Dense understory up to 2 m** can produce false stems or hide real ones (partly helped by the search from above, the stem checks, the cylinder check and the height curve).
- **Strongly leaning and multi-stemmed trees:** the assumption “stem = vertical pole” breaks; the lean correction is an approximation. A second stem is split only when it can be tracked for at least 4 m.
- **Shrubs:** thresholds (3 layers, 0.8 m, 0.5 m²) set on one plot; in a dense thicket they may join several shrubs into one or take young trees for shrubs.
- **DBH is not extended downwards** when the stem is hidden at 1.3 m (deliberately: extending gave large errors). Then stage 2 provides the DBH or it stays empty.
- **Species from shape** is preliminary; in a dense stand use “spruce only / beech only” or the tree review.
- **Wood / leaf split** not verified numerically.
- **Stem check, cylinder check and snags** are rules with thresholds set on a few plots; they can be wrong on unusual trees (e.g. very young spruces densely branched down to the ground).
- **Form factors** are approximate, to be replaced with regional tables.
- **Terrain model** is simple: good for terrestrial scans, not for airborne ones; tree height farther than 4 m from visible ground is uncertain.
- **GPS under a canopy** is off by several metres; the matching can be ambiguous (rating “uncertain”).
- **A large scan** (40 million points or more) needs about 8 GB of free memory.

13. References

Andrew A.M. (1979) Another efficient algorithm for convex hulls in two dimensions. Information Processing Letters 9(5): 216–219.

Barber C.B., Dobkin D.P., Huhdanpaa H. (1996) The Quickhull algorithm for convex hulls. ACM Transactions on Mathematical Software 22(4): 469–483.

Bentley J.L. (1975) Multidimensional binary search trees used for associative searching. Communications of the ACM 18(9): 509–517.

Besl P.J., McKay N.D. (1992) A method for registration of 3-D shapes. IEEE Transactions on Pattern Analysis and Machine Intelligence 14(2): 239–256.

Breiman L. (2001) Random forests. Machine Learning 45(1): 5–32.

Curtis R.O., Marshall D.D. (2000) Why quadratic mean diameter? Western Journal of Applied Forestry 15(3): 137–139.

Danielsson P.-E. (1980) Euclidean distance mapping. Computer Graphics and Image Processing 14(3): 227–248.

Delaunay B. (1934) Sur la sphère vide. Izvestia Akademii Nauk SSSR 7: 793–800.

Demantké J., Mallet C., David N., Vallet B. (2011) Dimensionality based scale selection in 3D lidar point clouds. ISPRS Archives XXXVIII-5/W12.

Dijkstra E.W. (1959) A note on two problems in connexion with graphs. Numerische Mathematik 1: 269–271.

Ester M., Kriegel H.-P., Sander J., Xu X. (1996) A density-based algorithm for discovering clusters in large spatial databases with noise. Proc. KDD-96: 226–231.

Fischler M.A., Bolles R.C. (1981) Random sample consensus. Communications of the ACM 24(6): 381–395.

Hackenberg J., Spiecker H., Calders K., Disney M., Raumonen P. (2015) SimpleTree – an efficient open source tool to build tree models from TLS clouds. Forests 6(11): 4245–4294.

Hopcroft J., Tarjan R. (1973) Algorithm 447: Efficient algorithms for graph manipulation. Communications of the ACM 16(6): 372–378.

Hoppe H., DeRose T., Duchamp T., McDonald J., Stuetzle W. (1992) Surface reconstruction from unorganized points. Computer Graphics 26(2): 71–78.

Horn B.K.P. (1981) Hill shading and the reflectance map. Proceedings of the IEEE 69(1): 14–47.

Husch B., Beers T.W., Kershaw J.A. (2003) Forest Mensuration. 4th ed., Wiley.

Jaccard P. (1912) The distribution of the flora in the alpine zone. New Phytologist 11(2): 37–50.

Kabsch W. (1976) A solution for the best rotation to relate two sets of vectors. Acta Crystallographica A32: 922–923.

Karney C.F.F. (2011) Transverse Mercator with an accuracy of a few nanometers. Journal of Geodesy 85(8): 475–485.

Knutsson H., Westin C.-F. (1993) Normalized and differential convolution. Proceedings CVPR 1993: 515–523.

Krishna Moorthy S.M., Calders K., Vicari M.B., Verbeeck H. (2020) Improved supervised learning-based approach for leaf and wood classification from LiDAR point clouds of forests. IEEE Transactions on Geoscience and Remote Sensing 58(5): 3057–3070.

Krüger L. (1912) Konforme Abbildung des Erdellipsoids in der Ebene. Veröffentlichung des Königlich Preussischen Geodätischen Institutes, N.F. 52.

Kuhn H.W. (1955) The Hungarian method for the assignment problem. Naval Research Logistics Quarterly 2: 83–97.

Kåsa I. (1976) A circle fitting procedure and its error analysis. IEEE Transactions on Instrumentation and Measurement IM-25(1): 8–14.

Levenberg K. (1944) A method for the solution of certain non-linear problems in least squares. Quarterly of Applied Mathematics 2: 164–168.

Lorensen W.E., Cline H.E. (1987) Marching cubes: a high resolution 3D surface construction algorithm. Computer Graphics 21(4): 163–169.

Marquardt D.W. (1963) An algorithm for least-squares estimation of nonlinear parameters. Journal of the Society for Industrial and Applied Mathematics 11(2): 431–441.

Näslund M. (1936) Skogsförsöksanstaltens gallringsförsök i tallskog. Meddelanden från Statens Skogsförsöksanstalt 29.

Raumonen P., Kaasalainen M., Åkerblom M., Kaasalainen S., Kaartinen H., Vastaranta M., Holopainen M., Disney M., Lewis P. (2013) Fast automatic precision tree models from terrestrial laser scanner data. Remote Sensing 5(2): 491–520.

Rusu R.B., Cousins S. (2011) 3D is here: Point Cloud Library (PCL). IEEE ICRA 2011.

Rusu R.B., Marton Z.C., Blodow N., Dolha M., Beetz M. (2008) Towards 3D point cloud based object maps for household environments. Robotics and Autonomous Systems 56(11): 927–941.

Serra J. (1982) Image Analysis and Mathematical Morphology. Academic Press.

Tao S., Wu F., Guo Q. et al. (2015) Segmenting tree crowns from terrestrial and mobile LiDAR data by exploring ecological theories. ISPRS Journal of Photogrammetry and Remote Sensing 110: 66–76.

Umeyama S. (1991) Least-squares estimation of transformation parameters between two point patterns. IEEE TPAMI 13(4): 376–380.

Vicari M.B., Disney M., Wilkes P., Burt A., Calders K., Woodgate W. (2019) Leaf and wood classification framework for terrestrial LiDAR point clouds. Methods in Ecology and Evolution 10(5): 680–694.

Wang D. (2020) Unsupervised semantic and instance segmentation of forest point clouds. ISPRS Journal of Photogrammetry and Remote Sensing 165: 86–97.

Wang D., Momo Takoudjou S., Casella E. (2020) LeWoS: A universal leaf-wood classification method to facilitate the 3D modelling of large tropical trees using terrestrial LiDAR. Methods in Ecology and Evolution 11(3): 376–389.

Weinmann M., Jutzi B., Hinz S., Mallet C. (2015) Semantic point cloud interpretation based on optimal neighborhoods, relevant features and efficient classifiers. ISPRS Journal of Photogrammetry and Remote Sensing 105: 286–304.

West P.W. (2009) Tree and Forest Measurement. 2nd ed., Springer.

Yarowsky D. (1995) Unsupervised word sense disambiguation rivaling supervised methods. Proc. ACL 1995: 189–196.

The references were chosen for the algorithms actually used in the code. The publication details of Näslund 1936, Delaunay 1934 and Tao et al. 2015 are worth checking before citing them in scientific work.